Embarcatech - 3ª Fase | Instituto Hardware BR

Controle de Ambientação Inteligente

Documentação Técnica

Equipe Campinas: Gabriel Mattano da Silva | Jorge Wilker Mamede de Andrade | Roger de Lima Araujo de Melo

Versão: 1.0

Data: Janeiro 2026

Projeto: Sistema de Monitoramento Ambiental com ESP32

Plataforma: ESP32 DevKit C (ESP32-WROOM-32)

Framework: Arduino (via PlatformIO)

Sumário

1. Visão Geral
2. Requisitos
3. Decisões de Design (ADRs)
4. Arquitetura do Sistema
5. Visão de Módulos
6. Modelo de Dados
7. APIs e Interfaces
8. Fluxos de Operação
9. Especificações de Hardware
10. Especificações de Software
11. Padrões de Código
12. Configuração e Instalação
13. Guia de Deploy e Operação
14. Protocolos e Comunicação

15. Calibração e Configuração
16. Especificações Técnicas Detalhadas
17. Diagramas de Conexão
18. Troubleshooting

---

1. Visão Geral

1.1 Descrição do Sistema

Este projeto implementa um sistema de controle ambiental distribuído composto por 4 módulos ESP32 que se comunicam via rede mesh WiFi. O sistema coleta dados ambientais (temperatura, umidade, luminosidade) de sensores internos e externos, processa essas informações no módulo Gateway, e controla atuadores (iluminação, irrigação, umidificação, ventilação) baseado em regras configuráveis. Todos os dados são transmitidos para a plataforma Blynk Cloud para visualização e controle remoto.

1.2 Objetivos

- Monitoramento contínuo de variáveis ambientais (internas e externas)
- Comunicação distribuída via rede mesh WiFi entre módulos
- Transmissão de dados em tempo real para Blynk Cloud
- Controle automático de atuadores baseado em regras configuráveis
- Integração com plataforma IoT (Blynk) para visualização e controle remoto
- Operação autônoma com reconexão automática
- Interface remota para visualização de dados e controle manual

1.3 Escopo

O sistema é composto pelos seguintes módulos:

Módulo Sensor Interno: - Leitura de sensores I2C (AHT20/AHT21, BH1750) - Leitura de sensor analógico (umidade do solo capacitivo) - Comunicação via rede mesh com Gateway

Módulo Sensor Externo: - Leitura de sensores I2C (AHT20/AHT21, BH1750) - Comunicação via rede mesh com Gateway

Módulo Gateway: - Comunicação WiFi com reconexão automática - Integração com Blynk Cloud - Armazenamento local (SD Card) - Coordenação entre módulos - Monitoramento de qualidade de sinal WiFi (RSSI)

Módulo Atuador: - Controle de iluminação LED (PWM) - Controle de solenóide de irrigação - Controle de nebulizador ultrassônico - Controle de ventiladores (entrada e exaustão) - Comunicação via rede mesh com Gateway

---

2. Requisitos

2.1 Requisitos Funcionais

- **RF01 - Leitura de sensores internos:** Leitura contínua de temperatura, umidade do ar, luminosidade e umidade do solo. (Prioridade: Alta, Status: Implementado)
- **RF02 - Transmissão de dados ao Blynk:** Envio de dados de sensores para servidor Blynk Cloud em tempo real. (Prioridade: Alta, Status: Implementado)
- **RF03 - Reconexão WiFi automática:** Gerenciador de conexão em segundo plano, reconecta automaticamente em caso de queda. (Prioridade: Alta, Status: Implementado)
- **RF04 - Operação offline:** Sistema continua funcionando mesmo sem conexão Blynk (leituras locais). (Prioridade: Alta, Status: Implementado)
- **RF05 - Monitoramento de qualidade WiFi:** Leitura e transmissão do nível de sinal WiFi (RSSI). (Prioridade: Média, Status: Implementado)
- **RF06 - Logging via Serial Monitor:** Exibição de dados e status no Serial Monitor para depuração. (Prioridade: Média, Status: Implementado)

Nota: Este sistema implementa um subconjunto dos requisitos do projeto maior (sistema de controle de estufa). Os requisitos acima referem-se especificamente ao módulo Gateway de monitoramento de sensores.

2.2 Requisitos Não Funcionais

- **RNF01 - Tempo de leitura de sensores:** ≤ 2 segundos (Categoria: Desempenho, Prioridade: Alta, Status: Atendido)
- **RNF02 - Precisão de temperatura:** ±0.3°C (Categoria: Precisão, Prioridade: Alta, Status: Atendido)
- **RNF03 - Precisão de umidade:** ±2% RH (Categoria: Precisão, Prioridade: Alta, Status: Atendido)
- **RNF04 - Precisão de luminosidade:** ±20% (típico) (Categoria: Precisão, Prioridade: Média, Status: Atendido)
- **RNF05 - Reconexão WiFi automática:** < 1 minuto (Categoria: Robustez, Prioridade: Alta, Status: Atendido)
- **RNF06 - Operação offline funcional:** Funcional sem internet (Categoria: Resiliência, Prioridade: Alta, Status: Atendido)
- **RNF07 - Interface intuitiva:** Via Blynk App (Categoria: Usabilidade, Prioridade: Média, Status: Atendido)
- **RNF08 - Temperatura de operação:** 0°C a 70°C (Categoria: Robustez ambiental, Prioridade: Média, Status: Atendido)
- **RNF09 - Consumo de energia:** ~80-120 mA @ 3.3V (Categoria: Eficiência energética, Prioridade: Média, Status: Atendido)
- **RNF10 - Compatibilidade WiFi:** 802.11 b/g/n (2.4 GHz) (Categoria: Compatibilidade, Prioridade: Alta, Status: Atendido)

2.3 Rastreabilidade de Requisitos

- **Monitoramento ambiental em tempo real:** RF01, RF02, RNF01
- **Operação confiável e robusta:** RF03, RF04, RNF05, RNF06
- **Interface remota intuitiva:** RF02, RNF07
- **Baixo consumo de energia:** RNF09
- **Compatibilidade com redes domésticas:** RNF10

---

3. Decisões de Design (ADRs)

ADR-001: Uso do Framework Arduino

Status: Aceito
Data: Novembro 2025
Contexto: Necessidade de escolher um framework para desenvolvimento no ESP32.

Decisão: Utilizar Arduino Framework via PlatformIO.

Consequências: - Grande ecossistema de bibliotecas disponíveis - Código mais legível e fácil de manter - Compatibilidade com bibliotecas populares (Adafruit, Blynk) - Reduz curva de aprendizado para novos desenvolvedores - Menor controle sobre recursos de baixo nível comparado ao ESP-IDF - Overhead maior que programação bare-metal

Alternativas consideradas: - ESP-IDF: Mais controle, mas curva de aprendizado maior - MicroPython: Mais fácil, mas performance inferior

---

ADR-002: Plataforma IoT - Blynk

Status: Aceito
Data: Novembro 2025 **Contexto:** Necessidade de plataforma para visualização remota de dados.

Decisão: Utilizar Blynk Cloud como plataforma IoT.

Consequências: - Interface gráfica disponível - Biblioteca oficial com suporte ativo - Suporte nativo para ESP32 - Virtual Pins facilitam comunicação - Dependência de serviço externo (Blynk Cloud) - Limitações na versão gratuita

Alternativas consideradas: - ThingSpeak: Mais focado em analytics, menos intuitivo - Firebase: Mais complexo, requer mais configuração - Servidor próprio: Mais controle, mas maior complexidade

---

ADR-003: Barramento I2C Compartilhado

Status: Aceito
Data: Novembro 2025
Contexto: Múltiplos sensores I2C (AHT20/AHT21 e BH1750) precisam ser conectados.

Decisão: Compartilhar o barramento I2C usando endereços diferentes.

Consequências: - Economiza pinos GPIO - Simplifica conexões físicas - Padrão comum em projetos IoT - Requer atenção aos endereços I2C (evitar conflitos) - Limitação de distância do barramento

Alternativas consideradas: - Barramentos I2C separados: Mais pinos, mais complexidade - SPI: Mais rápido, mas mais pinos necessários

---

ADR-004: Sensor Analógico para Umidade do Solo

Status: Aceito
Data: Novembro 2025
Contexto: Necessidade de medir umidade do solo de forma econômica.

Decisão: Utilizar sensor capacitivo analógico conectado ao ADC do ESP32.

Consequências: - Custo baixo - Simplicidade de conexão (apenas 3 fios) - Calibração via software - Requer calibração específica por sensor - Menor precisão que sensores digitais profissionais

Alternativas consideradas: - Sensor digital I2C: Mais preciso, mas mais caro - Sensor de tensão: Mais complexo, requer circuito adicional

---

ADR-005: Intervalo de Leitura de 2 Segundos

Status: Aceito
Data: Novembro 2025 **Contexto:** Definir frequência de amostragem dos sensores.

Decisão: Leitura de sensores a cada 2 segundos (0.5 Hz).

Consequências: - Balanceamento entre responsividade e consumo - Adequado para monitoramento ambiental (variações lentas) - Reduz carga no servidor Blynk - Pode não capturar variações muito rápidas - Latência de até 2 segundos para detectar mudanças

Alternativas consideradas: - 1 segundo: Mais responsivo, mas maior consumo e carga - 5 segundos: Menor consumo, mas menos responsivo

---

ADR-006: Operação Graceful Degradation

Status: Aceito
Data: Novembro 2025
Contexto: Sistema deve funcionar mesmo com falhas parciais.

Decisão: Sistema continua operando mesmo se Blynk não conectar, mantendo leituras locais.

Consequências: - Maior robustez e confiabilidade - Sistema útil mesmo sem internet - Dados disponíveis via Serial Monitor - Perda de visualização remota em caso de falha - Dados não são persistidos localmente (sem SD Card)

Alternativas consideradas: - Falha total se Blynk não conectar: Menos robusto - Persistência em SD Card: Mais complexo, requer hardware adicional

---

4. Arquitetura do Sistema

4.1 Componentes Principais

O sistema é composto por 4 módulos principais que se comunicam via rede mesh WiFi:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                Sistema de Controle Ambiental                             │
│                                                                          │
│  ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐     │
│  │   Módulo         │   │   Módulo         │   │   Módulo         │     │
│  │  Sensor          │   │   Sensor         │   │   Gateway        │     │
│  │   Interno        │   │   Externo        │   │                  │     │
│  │                  │   │                  │   │                  │     │
│  │ ESP32 +          │   │ ESP32 +          │   │ ESP32 +          │     │
│  │ Sensores         │   │ Sensores         │   │ WiFi +           │     │
│  │ Internos         │   │ Externos         │   │ Blynk +          │     │
│  │                  │   │                  │   │ SD Card          │     │
│  └──────────────────┘   └──────────────────┘   └──────────────────┘     │
│        │                    │                    │                      │
│        └────────────────────┴────────────────────┘                      │
│                             │                                            │
│                             ▼                                            │
│                      ┌──────────────┐                                   │
│                      │   Rede       │                                   │
│                      │   Mesh       │                                   │
│                      │   WiFi       │                                   │
│                      └──────────────┘                                   │
│                             │                                            │
│                             ▼                                            │
│                      ┌──────────────┐                                   │
│                      │   Módulo     │                                   │
│                      │   Atuador    │                                   │
│                      │              │                                   │
│                      │ ESP32 +      │                                   │
│                      │ Drivers +    │                                   │
│                      │ Atuadores    │                                   │
│                      └──────────────┘                                   │
└──────────────────────────────────────────────────────────────────────────┘
```

4.2 Fluxo de Dados

1. **Aquisição**: Módulos Sensor Interno e Sensor Externo coletam dados a cada 2 segundos
2. **Transmissão Mesh**: Dados enviados via rede mesh para o módulo Gateway
3. **Processamento**: Gateway processa e converte valores
4. **Transmissão Cloud**: Gateway envia dados via WiFi para Blynk Cloud

5. **Controle**: Gateway envia comandos para módulo Atuador baseado em regras
6. **Visualização**: Dados exibidos no dashboard Blynk
7. **Logging**: Dados armazenados localmente (SD Card) e exibidos no Serial Monitor

---

5. Visão de Módulos

5.1 Estrutura Modular do Sistema

O sistema é composto pelos seguintes módulos funcionais:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                Sistema de Controle Ambiental                                 │
│                                                                              │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐          │
│  │  Módulo          │  │  Módulo          │  │  Módulo          │          │
│  │ Sensor           │  │  Sensor          │  │  Gateway         │          │
│  │  Interno         │  │  Externo         │  │                  │          │
│  │                  │  │                  │  │                  │          │
│  │- AHT20/AHT21     │  │- AHT20/AHT21     │  │- WiFi            │          │
│  │- BH1750          │  │- BH1750          │  │- Blynk           │          │
│  │- Solo ADC        │  │                  │  │- Mesh            │          │
│  │                  │  │                  │  │- SD Card         │          │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘          │
│                                                                              │
│                    ┌──────────────────┐                                      │
│                    │  Módulo          │                                      │
│                    │  Atuador         │                                      │
│                    │                  │                                      │
│                    │- LED PWM         │                                      │
│                    │- Solenóide       │                                      │
│                    │- Nebulizador     │                                      │
│                    │- Ventilador      │                                      │
│                    │- Exaustor        │                                      │
│                    └──────────────────┘                                      │
└──────────────────────────────────────────────────────────────────────────────┘
```

5.2 Módulo Sensor Interno

Responsabilidade: Coletar dados ambientais do interior do ambiente controlado (estufa).

Componentes: - **Sensor AHT20/AHT21**: Temperatura e umidade do ar interno - **Sensor BH1750**: Luminosidade ambiente interna - **Sensor de Solo**: Umidade do solo (via ADC) - **MCU ESP32**: Processamento e comunicação

Dados Coletados: - Temperatura interna (°C) - Umidade relativa interna (% RH) - Luminosidade interna (lux) - Umidade do solo (%)

Interface:

```
// Inicialização
bool ahtInitialized = aht.begin();
bool bh1750Initialized = lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);

// Leitura
aht.getEvent(&humid, &temp);
lightLevel = lightMeter.readLightLevel();
soilMoistureRaw = analogRead(SOIL_MOISTURE_PIN);
```

Dependências: - Biblioteca `Adafruit_AHTX0` - Biblioteca `BH1750` - Biblioteca `Wire` (I2C) - Hardware I2C (GPIO21, GPIO22) - ADC (GPIO34) - Comunicação via rede mesh com Gateway

Comunicação: - Envia dados periodicamente para o módulo Gateway via rede mesh - Recebe comandos de configuração do Gateway

---

5.3 Módulo Sensor Externo

Responsabilidade: Coletar dados ambientais do exterior do ambiente controlado para referência.

Componentes: - **Sensor AHT20/AHT21**: Temperatura e umidade do ar externo - **Sensor BH1750**: Luminosidade ambiente externa - **MCU ESP32**: Processamento e comunicação

Dados Coletados: - Temperatura externa (°C) - Umidade relativa externa (% RH) - Luminosidade externa (lux)

Interface:

```
// Inicialização similar ao sensor interno
bool ahtInitialized = aht.begin();
bool bh1750Initialized = lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);

// Leitura
aht.getEvent(&humid, &temp);
lightLevel = lightMeter.readLightLevel();
```

Dependências: - Biblioteca Adafruit_AHTX0 - Biblioteca BH1750 - Biblioteca Wire (I2C) - Hardware I2C (GPIO21, GPIO22) - Comunicação via rede mesh com Gateway

Comunicação: - Envia dados periodicamente para o módulo Gateway via rede mesh - Recebe comandos de configuração do Gateway

---

5.4 Módulo Gateway

Responsabilidade: Gerenciar comunicação entre módulos, integração com Blynk Cloud, armazenamento local e coordenação do sistema.

Componentes: - **MCU ESP32**: Processamento central - **WiFi Manager**: Conexão e reconexão WiFi - **Blynk Client**: Comunicação com Blynk Cloud - **Rede Mesh**: Comunicação com módulos sensores e atuador - **SD Card**: Armazenamento local de dados (opcional) - **RTC**: Relógio em tempo real para timestamp (opcional)

Funcionalidades: - Recebe dados dos módulos Sensor Interno e Sensor Externo - Transmite dados para Blynk Cloud - Envia comandos para módulo Atuador baseado em regras de controle - Armazena logs localmente (se SD Card disponível) - Gerencia sincronização de tempo entre módulos

Interface:

```
// WiFi
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
WiFi.status() == WL_CONNECTED

// Blynk
Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
Blynk.connected()
Blynk.virtualWrite(V0, value);
Blynk.run();

// Mesh (exemplo)
mesh_send_data(sensor_data);
mesh_receive_command();
```

Dependências: - Biblioteca `WiFi` (built-in ESP32) - Biblioteca `BlynkSimpleEsp32` - Biblioteca de rede mesh (ESP-MESH ou similar) - Credenciais WiFi e Blynk - SD Card (opcional, para logging)

Comunicação: - Recebe dados dos módulos Sensor Interno e Sensor Externo - Envia comandos para módulo Atuador - Comunica com Blynk Cloud via WiFi

---

5.5 Módulo Atuador

Responsabilidade: Controlar atuadores ambientais (iluminação, irrigação, umidificação e ventilação) baseado em dados dos sensores e comandos remotos.

Componentes: - **MCU + Módulo Wi-Fi**: Unidade de controle principal (ESP32) - **Driver de Iluminação LED (PWM)**: Controle de intensidade da fita de LED - **Driver de Solenóide (MOSFET + Relé)**: Controle do solenóide de irrigação - **Driver de Nebulizador (MOSFET)**: Controle do nebulizador ultrassônico - **Driver Motor para Fan (PWM)**: Controle de velocidade dos ventiladores - **Fonte de Alimentação**: Alimentação do sistema - **Antena Wi-Fi**: Comunicação sem fio

Atuadores Controlados: - **Fita de LED**: Iluminação ajustável via PWM - **Solenóide para Irrigação**: Controle de irrigação do solo - **Nebulizador Ultrassônico**: Umidificação do ambiente - **Fan para Ventilação**: Ventilador de entrada de ar - **Fan para Exaustão**: Ventilador de saída de ar

Interface:

```
// Controle PWM para LED
analogWrite(LED_PIN, brightness);  // 0-255

// Controle de solenóide (via relé)
digitalWrite(SOLENOID_PIN, HIGH);   // Liga irrigação
digitalWrite(SOLENOID_PIN, LOW);    // Desliga irrigação

// Controle de nebulizador (via MOSFET)
digitalWrite(NEBULIZER_PIN, HIGH);  // Liga umidificação
digitalWrite(NEBULIZER_PIN, LOW);   // Desliga umidificação

// Controle PWM para ventiladores
analogWrite(FAN_INTAKE_PIN, speed);   // 0-255
analogWrite(FAN_EXHAUST_PIN, speed);  // 0-255
```

Dependências: - Biblioteca `WiFi` (built-in ESP32) para comunicação - Biblioteca `BlynkSimpleEsp32` para comandos remotos - Circuitos drivers externos (MOSFET, Relé) - Fonte de alimentação adequada para atuadores - Sinais de controle dos sensores (para operação automática)

Diagrama do Módulo Atuador:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    Módulo Atuador                                        │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │          MCU + Módulo Wi-Fi (ESP32)                              │    │
│  │                                                                  │    │
│  │  ┌──────────────────┐          ┌──────────────────┐              │    │
│  │  │   Fonte de       │──────────│   Antena         │              │    │
│  │  │  Alimentação     │          │  Wi-Fi           │              │    │
│  │  └──────────────────┘          └──────────────────┘              │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│          │                                                               │
│  ┌───────┼──────┬──────┬──────┬──────┬──────┬──────┬──────┐             │
│  │       │      │      │      │      │      │      │      │             │
│  ▼       ▼      ▼      ▼      ▼      ▼      ▼      ▼      ▼             │
│ ┌──┐    ┌──┐   ┌──┐   ┌──┐   ┌──┐   ┌──┐   ┌──┐   ┌──┐                │
│ │LED│   │Solen.│ │Nebul.│ │Fan│  │Fan│  │Fan│  │   │                    │
│ │PWM│   │MOSFET│ │MOSFET│ │PWM│  │PWM│  │PWM│  │   │                    │
│ │Drv.│  │+Relé│  │Drv.│  │Drv.│  │Drv.│  │   │                         │
│ └──┘    └──┘   └──┘   └──┘   └──┘   └──┘   └──┘                         │
│  │       │      │      │      │      │      │      │                    │
│  ▼       ▼      ▼      ▼      ▼      ▼      ▼      ▼                    │
│ ┌──┐    ┌──┐   ┌──┐   ┌──┐   ┌──┐   ┌──┐   ┌──┐                        │
│ │Fita│  │Solen.│ │Nebul.│ │Fan│  │Fan│  │   │                          │
│ │LED│   │Irrig.│ │Ultr.│  │Vent.│ │Exaust│ │   │                        │
│ └──┘    └──┘   └──┘   └──┘   └──┘   └──┘   └──┘                         │
└──────────────────────────────────────────────────────────────────────────┘
```

Fluxo de Controle:

1. **Entrada de Dados:**
   - Recebe dados dos sensores via rede mesh ou Blynk
   - Recebe comandos manuais via Blynk dashboard
2. **Processamento:**
   - Compara valores dos sensores com setpoints configurados
   - Aplica lógica de controle (histerese, anti-oscilação)
   - Determina estados dos atuadores

3. **Saída de Controle:**
   - Envia sinais PWM para controle de velocidade/intensidade
   - Envia sinais digitais para ligar/desligar atuadores
   - Através dos drivers, controla os atuadores físicos

Características de Segurança: - Estados seguros em caso de falha (atuadores desligados) - Proteção contra sobrecorrente nos drivers - Validação de comandos antes de executar - Watchdog timer para reinicialização em caso de travamento

---

5.6 Diagrama de Dependências entre Módulos

```
┌──────────────────────────────┐
│   Módulo Gateway             │
│  (Coordenação)               │
└──────────────────────────────┘
            │
    ┌───────┴──────┬──────────────────┐
    │      │       │                  │
┌───▼───┐ ┌───▼───┐ ┌───▼───┐   ┌───▼───┐
│Sensor │ │Sensor │ │Atuador│   │ Blynk │
│Interno│ │Externo│ │       │   │ Cloud │
└───┬───┘ └───┬───┘ └───┬───┘   └───────┘
    │         │         │
    │     ┌───▼─────────▼───┐
    │     │   Rede Mesh     │
    │     │   (WiFi)        │
    │     └─────────────────┘
    │
┌───▼──────────────┐
│ Hardware I2C     │
│ Hardware ADC     │
│ (Sensores)       │
└──────────────────┘
```

---

6. Modelo de Dados

6.1 Estruturas de Dados Principais

6.1.1 Dados dos Sensores

```
// Variáveis globais de leitura
float temperature = 0.0;            // Temperatura em °C
float humidity = 0.0;               // Umidade do ar em % RH
float lightLevel = 0.0;             // Luminosidade em lux
int soilMoistureRaw = 0;            // Valor ADC bruto (0-4095)
float soilMoisturePercent = 0.0;    // Umidade do solo em % (0-100)
int wifiRSSI = 0;                   // Sinal WiFi em dBm
```

Tipos e Faixas:

- **temperature:** float | -40.0 a +85.0 °C | Precisão: 0.01°C
- **humidity:** float | 0.0 a 100.0 % RH | Precisão: 0.01%
- **lightLevel:** float | 1.0 a 65535.0 lux | Precisão: 1 lux
- **soilMoistureRaw:** int | 0 a 4095 ADC | Precisão: 1 LSB
- **soilMoisturePercent:** float | 0.0 a 100.0 % | Precisão: 0.1%
- **wifiRSSI:** int | -100 a 0 dBm | Precisão: 1 dBm

6.1.2 Status do Sistema

```
// Status de inicialização dos sensores
bool ahtInitialized = false;        // Status do sensor AHT20/AHT21
bool bh1750Initialized = false;     // Status do sensor BH1750
```

Estados Possíveis:

- **ahtInitialized (bool):** true / false - Sensor AHT20/AHT21 inicializado com sucesso
- **bh1750Initialized (bool):** true / false - Sensor BH1750 inicializado com sucesso

- **WiFi.status() (wl_status_t):** WL_CONNECTED / outros - Status da conexão WiFi
- **Blynk.connected() (bool):** true / false - Status da conexão Blynk

---

6.1.3 Parâmetros de Configuração

```
// Configuração de pinos
#define SOIL_MOISTURE_PIN 34          // GPIO para sensor de solo

// Calibração do sensor de solo
const int SOIL_DRY_VALUE = 2521;     // Valor ADC quando seco
const int SOIL_WET_VALUE = 1200;      // Valor ADC quando molhado

// Intervalos de tempo (em milissegundos)
const unsigned long sensorReadInterval = 2000;        // 2 segundos
const unsigned long connectionCheckInterval = 2000;   // 2 segundos
const unsigned long blynkTimeout = 30000;             // 30 segundos
```

6.1.4 Controle de Timing

```
// Timestamps para controle de intervalos
unsigned long lastConnectionCheck = 0;  // Última verificação de conexão
unsigned long lastSensorRead = 0;       // Última leitura de sensores
unsigned long blynkStartTime = 0;       // Início da tentativa de conexão Blynk
```

6.2 Formato de Dados para Blynk

6.2.1 Virtual Pins Mapping

- **V0:** temperature (Float) - Formato: XX.X (Ex: 25.3)
- **V1:** humidity (Float) - Formato: XX.X (Ex: 60.5)
- **V2:** lightLevel (Float) - Formato: XXXX (Ex: 350.0)

- **V3:** soilMoisturePercent (Float) - Formato: XX.X (Ex: 45.0)
- **V4:** wifiRSSI (Integer) - Formato: -XX (Ex: -45)

6.2.2 Protocolo de Envio

```
// Formato de envio para Blynk
Blynk.virtualWrite(V0, temperature);          // Float
Blynk.virtualWrite(V1, humidity);             // Float
Blynk.virtualWrite(V2, lightLevel);           // Float
Blynk.virtualWrite(V3, soilMoisturePercent);  // Float
Blynk.virtualWrite(V4, wifiRSSI);             // Integer
```

Nota: Blynk aceita valores numéricos (float ou int) diretamente. A formatação visual é feita no dashboard.

6.3 Formato de Dados para Serial Monitor

6.3.1 Estrutura de Log

```
=== Sistema de Monitoramento ESP32 ===

Sensor de Umidade do Solo Capacitivo:
  Configurado no pino GPIO34
  [OK] Pronto para leitura analógica

Inicializando sensor AHT20/AHT21...
[OK] Sensor AHT20/AHT21 inicializado com sucesso.

--- Leituras dos Sensores ---
Temperatura: 25.3 °C
Umidade Ar: 60.5 %
Luminosidade: 350 lux
Umidade Solo: 45 % (ADC: 1860)
Sinal WiFi: -45 dBm (Excelente)
```

6.3.2 Codificação de Caracteres

- **Encoding:** UTF-8
- **Velocidade:** 115200 baud
- **Formato:** Texto plano com quebras de linha (\n)

6.4 Validação e Limites

6.4.1 Validação de Dados

```
// Limitação de faixa para umidade do solo
if (soilMoisturePercent < 0) soilMoisturePercent = 0;
if (soilMoisturePercent > 100) soilMoisturePercent = 100;
```

6.4.2 Valores de Erro/Indefinidos

- **temperature:** 0.0 (Sensor não inicializado)
- **humidity:** 0.0 (Sensor não inicializado)
- **lightLevel:** 0.0 (Sensor não inicializado)
- **soilMoisturePercent:** 0.0 (Falha na leitura ADC)
- **wifiRSSI:** 0 ou não atualizado (WiFi desconectado)

Nota: O sistema não envia valores para Blynk se o sensor não estiver inicializado, evitando dados incorretos.

---

7. APIs e Interfaces

7.1 Interface Blynk (API de Comunicação)

O sistema utiliza a biblioteca Blynk como interface principal para comunicação com a nuvem. A biblioteca Blynk implementa um protocolo proprietário sobre TCP/IP.

7.1.1 Inicialização

```
Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
```

Parâmetros: - BLYNK_AUTH_TOKEN: Token de autenticação do projeto Blynk (string) - WIFI_SSID: Nome da rede WiFi (string) - WIFI_PASSWORD: Senha da rede WiFi (string)

Retorno: Nenhum (void)

Comportamento: - Tenta conectar ao WiFi usando as credenciais fornecidas - Após conexão WiFi, tenta conectar ao servidor Blynk Cloud - Timeout de 30 segundos para conexão Blynk

---

7.1.2 Verificação de Conexão

```
bool connected = Blynk.connected();
```

Retorno: bool - true: Conectado ao Blynk Cloud - false: Desconectado ou não conectado

Uso:

```
if (Blynk.connected()) {
  // Enviar dados apenas se conectado
  Blynk.virtualWrite(V0, temperature);
}
```

7.1.3 Envio de Dados (Virtual Pins)

```
Blynk.virtualWrite(V0, temperature);
Blynk.virtualWrite(V1, humidity);
Blynk.virtualWrite(V2, lightLevel);
Blynk.virtualWrite(V3, soilMoisturePercent);
Blynk.virtualWrite(V4, wifiRSSI);
```

Parâmetros: - V0-V4: Número do Virtual Pin (0-255) - Segundo parâmetro: Valor a ser enviado (float ou int)

Retorno: Nenhum (void)

Comportamento: - Envia valor para o Virtual Pin especificado - Funciona apenas se Blynk.connected() retornar true - Não bloqueia a execução (assíncrono)

Virtual Pins Utilizados:

- **V0 (Temperatura):** Float - Temperatura em °C
- **V1 (Umidade Ar):** Float - Umidade relativa em %
- **V2 (Luminosidade):** Float - Luminosidade em lux
- **V3 (Umidade Solo):** Float - Umidade do solo em %
- **V4 (RSSI WiFi):** Integer - Sinal WiFi em dBm

7.1.4 Manutenção da Conexão

```
Blynk.run();
```

Descrição: Deve ser chamado regularmente no loop principal para manter a conexão ativa e processar mensagens recebidas.

Frequência Recomendada: A cada iteração do loop (ou pelo menos a cada 100ms)

Comportamento: - Processa mensagens recebidas do servidor - Mantém “heartbeat” da conexão - Detecta desconexões

---

7.2 Interface WiFi (ESP32)

7.2.1 Inicialização

```
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

Parâmetros: - WIFI_STA: Modo Station (cliente WiFi) - WIFI_SSID: Nome da rede (string) - WIFI_PASSWORD: Senha da rede (string)

7.2.2 Verificação de Status

```
wl_status_t status = WiFi.status();
```

Retorno: wl_status_t (enum)

Valores Possíveis: - WL_CONNECTED: Conectado à rede - WL_DISCONNECTED: Desconectado - WL_CONNECT_FAILED: Falha na conexão - WL_NO_SSID_AVAIL: SSID não encontrado - WL_IDLE_STATUS: Em processo de conexão

Uso:

```
if (WiFi.status() == WL_CONNECTED) {
  // WiFi conectado
}
```

7.2.3 Obtenção de Informações

```
IPAddress ip = WiFi.localIP();      // IP local
int rssi = WiFi.RSSI();             // Sinal em dBm
```

Retorno: - WiFi.localIP(): IPAddress (ex: 192.168.1.100) - WiFi.RSSI(): int (ex: -45)

7.3 Interface Serial (Logging)

7.3.1 Inicialização

```
Serial.begin(115200);
```

Parâmetros: - 115200: Velocidade em baud

7.3.2 Envio de Dados

```
Serial.print("Texto");
Serial.println("Texto com quebra de linha");
```

```
Serial.print(variable);
Serial.print(variable, 1);  // 1 casa decimal
```

Formatação: - Serial.print(): Sem quebra de linha - Serial.println(): Com quebra de linha (\n) - Serial.print(value, decimals): Formatação de casas decimais

---

7.4 Interface I2C (Sensores)

7.4.1 Inicialização

```
Wire.begin();
```

Descrição: Inicializa o barramento I2C nos pinos padrão (SDA=GPIO21, SCL=GPIO22)

7.4.2 Uso via Bibliotecas

O sistema não utiliza diretamente a interface Wire, mas sim bibliotecas de alto nível:

- **Adafruit_AHTX0**: Gerencia comunicação I2C com AHT20/AHT21
- **BH1750**: Gerencia comunicação I2C com BH1750

Interface das Bibliotecas:

```
// AHT20/AHT21
Adafruit_AHTX0 aht;
aht.begin();                        // Inicialização
aht.getEvent(&humid, &temp);        // Leitura

// BH1750
BH1750 lightMeter;
lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);  // Inicialização
lightLevel = lightMeter.readLightLevel();            // Leitura
```

7.5 Interface ADC (Sensor de Solo)

7.5.1 Leitura Analógica

```
int value = analogRead(SOIL_MOISTURE_PIN);
```

Parâmetros: - SOIL_MOISTURE_PIN: GPIO 34 (ADC1_CH6)

Retorno: int (0 a 4095)

Resolução: 12 bits (4096 níveis)

Tensão de Referência: 3.3V

Conversão:

```
float voltage = (value / 4095.0) * 3.3;  // Volts
```

---

8. Fluxos de Operação

8.1 Fluxo de Inicialização (Setup)

```
┌──────────────────┐
│   Início         │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Inicializa       │
│ Serial (115200)  │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Inicializa I2C   │
│ (Wire.begin())   │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Configura GPIO   │
│ 34 (ADC)         │
└──────────────────┘
        │
        ▼
```

```
┌──────────────────┐
│ Inicializa       │
│ AHT20/AHT21      │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Inicializa       │
│ BH1750           │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Conecta WiFi     │
│ (até 10s)        │
└──────────────────┘
        │
        ├──── Falha ──▶ [Sistema para]
        │
        ▼
    ┌──────────┐
    │ Sucesso│
    └──────────┘
        │
        ▼
┌──────────────────┐
│ Conecta Blynk    │
│ (até 30s)        │
└──────────────────┘
        │
        ├──── Falha ──▶ [Continua sem Blynk]
        │
        ▼
    ┌──────────┐
    │ Sucesso│
    └──────────┘
        │
        ▼
┌──────────────────┐
│ Sistema Pronto   │
│ Entra no Loop    │
└──────────────────┘
```

8.2 Fluxo Principal de Operação (Loop)

```
┌──────────────────┐
│   Loop Início    │
└──────────────────┘
        │
```

```
        ▼
┌──────────────────┐
│ WiFi conectado? │
└──────────────────┘
        │
        ├──── Não ──▶ [Tenta reconectar] ──▶ [Aguarda 2s]
        │
        ▼
    ┌─────────┐
    │   Sim   │
    └─────────┘
        │
        ▼
┌──────────────────┐
│ Blynk.run()      │
│ (mantém conexão) │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Passou 2s desde  │
│ última leitura?  │
└──────────────────┘
        │
        ├──── Não ──▶ [Aguarda 100ms] ──▶ [Volta ao início]
        │
        ▼
    ┌─────────┐
    │   Sim   │
    └─────────┘
        │
        ▼
┌──────────────────┐
│ Lê AHT20/AHT21   │
│ (temp, umidade)  │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Lê BH1750        │
│ (luminosidade)   │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Lê Sensor Solo   │
│ (ADC → %)        │
└──────────────────┘
        │
        ▼
```

```
┌──────────────────┐
│ Lê RSSI WiFi     │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Blynk conectado?│
└──────────────────┘
        │
        ├──── Sim ──▶ [Envia todos os dados via Virtual Pins]
        │
        ▼
┌──────────────────┐
│ Exibe no Serial  │
│ Monitor          │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│ Aguarda 100ms    │
└──────────────────┘
        │
        └────▶ [Volta ao início do loop]
```

8.3 Fluxo de Leitura de Sensores

```
┌──────────────────┐
│ Início Leitura   │
└──────────────────┘
        │
        ▼
┌────────────────────┐
│ AHT inicializado? │
└────────────────────┘
        │
        ├──── Sim ──▶ [Lê temperatura e umidade]
        │             [Atualiza variáveis globais]
        │
        ▼
┌──────────────────┐
│ BH1750 init?     │
└──────────────────┘
        │
        ├──── Sim ──▶ [Lê luminosidade]
        │             [Atualiza variável global]
        │
        ▼
┌──────────────────┐
│ Lê ADC GPIO34    │
```

```
│ (sensor solo)   │
└─────────────────┘
        │
        ▼
┌─────────────────────┐
│ Converte ADC        │
│ para %              │
│ (map + limites)     │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ Lê RSSI WiFi        │
│ (se conectado)      │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ Fim da Leitura      │
└─────────────────────┘
```

8.4 Fluxo de Reconexão WiFi

```
┌─────────────────────┐
│ WiFi desconectado   │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ Exibe mensagem      │
│ no Serial           │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│   WiFi.begin()      │
│ (tenta conectar)    │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ Aguarda 2s          │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ Verifica status     │
└─────────────────────┘
        │
```

```
├─── Conectado ──▶ [Continua operação normal]
│
└─── Desconectado ──▶ [Repete tentativa no próximo loop]
```

---

9. Especificações de Hardware

9.1 Microcontrolador

- **Modelo:** ESP32 DevKit C (ESP32-WROOM-32)
- **Chip:** ESP32-WROOM-32
- **CPU:** Dual-core Xtensa LX6 @ 240 MHz
- **RAM:** 520 KB SRAM
- **Flash:** 4 MB (varia conforme modelo)
- **WiFi:** 802.11 b/g/n (2.4 GHz)
- **GPIO:** 25 pinos expostos no header (maioria utilizável)
- **ADC:** 12-bit, 0-3.3V
- **Interface I2C:** Hardware I2C (SDA=GPIO21, SCL=GPIO22)

9.2 Sensores

9.2.1 Sensor AHT20/AHT21 (Temperatura e Umidade do Ar)

- **Tipo:** Sensor digital I2C
- **Interface:** I2C (endereço: 0x38)
- **Faixa de Temperatura:** -40°C a +85°C
- **Precisão de Temperatura:** ±0.3°C
- **Faixa de Umidade:** 0% a 100% RH
- **Precisão de Umidade:** ±2% RH
- **Tempo de Resposta:** < 8 segundos
- **Alimentação:** 2.2V a 5.5V (recomendado: 3.3V)
- **Consumo:** < 0.25 mA (modo medição)

9.2.2 Sensor BH1750 (Luminosidade)

- **Tipo:** Sensor digital I2C

- **Interface:** I2C (endereço: 0x23)
- **Faixa de Medição:** 1 a 65535 lux
- **Resolução:** 1 lux (modo HIGH_RES)
- **Precisão:** ±20% (típico)
- **Alimentação:** 2.4V a 3.6V (recomendado: 3.3V)
- **Consumo:** 120 µA (modo contínuo)
- **Modo de Operação:** CONTINUOUS_HIGH_RES_MODE

9.2.3 Sensor de Umidade do Solo Capacitivo

- **Tipo:** Sensor analógico capacitivo
- **Interface:** ADC (GPIO 34)
- **Faixa de Medição:** 0% a 100% (umidade relativa)
- **Resolução ADC:** 12-bit (0-4095)
- **Tensão de Alimentação:** 3.3V a 5V
- **Saída Analógica:** 0V a 3.3V (proporcional à umidade)
- **Calibração:** Valores específicos por sensor

Valores de Calibração (Este Projeto): - SOIL_DRY_VALUE = 2521 (solo seco, no ar) - SOIL_WET_VALUE = 1200 (solo molhado, na água)

9.3 Conexões de Hardware

9.3.1 Barramento I2C (Compartilhado)

- **GPIO 21:** SDA (Conectado a SDA do AHT20/AHT21 e BH1750)
- **GPIO 22:** SCL (Conectado a SCL do AHT20/AHT21 e BH1750)
- **3.3V:** VCC (Alimentação dos sensores)
- **GND:** GND (Terra comum)

Nota: Ambos os sensores compartilham o mesmo barramento I2C, utilizando endereços diferentes (0x38 para AHT20/AHT21, 0x23 para BH1750).

9.3.2 Sensor de Umidade do Solo

- **GPIO 34:** ADC/AOUT (Saída Analógica do Sensor)
- **3.3V:** VCC (Alimentação)
- **GND:** GND (Terra)

Importante: GPIO 34 é um pino apenas de entrada (input-only), ideal para leitura ADC.

---

10. Especificações de Software

10.1 Ambiente de Desenvolvimento

- **IDE:** Visual Studio Code com extensão PlatformIO
- **Framework:** Arduino Framework
- **Plataforma:** Espressif 32
- **Board:** esp32dev (compatível com ESP32 DevKit C)
- **Velocidade Serial:** 115200 baud

10.2 Bibliotecas e Dependências

- **adafruit/Adafruit AHTX0 (^2.0.5):** Comunicação com sensor AHT20/AHT21
- **claws/BH1750 (^1.3.0):** Comunicação com sensor BH1750
- **blynkkk/blynk-library (GitHub latest):** Integração com Blynk Cloud
- **WiFi (Built-in ESP32):** Comunicação WiFi
- **Wire (Built-in ESP32):** Comunicação I2C

10.3 Configuração do PlatformIO

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
upload_speed = 115200
monitor_speed = 115200
lib_deps =
```

```
adafruit/Adafruit AHTX0@^2.0.5
claws/BH1750@^1.3.0
https://github.com/blynkkk/blynk-library.git
```

Nota: A configuração esp32dev é compatível com ESP32 DevKit C (WROOM-32). O board esp32doit-devkit-v1 também funciona, mas esp32dev é mais genérico e compatível com a maioria das placas ESP32 baseadas em WROOM-32.

10.4 Estrutura do Código

```
src/
└── main.cpp            # Código principal da aplicação

include/
└── credentials.h       # Credenciais WiFi e Blynk (não versionado)
```

---

11. Padrões de Código

11.1 Convenções de Nomenclatura

11.1.1 Variáveis

- **Estilo:** camelCase
- **Globais:** Nomes descritivos, sem prefixos especiais
- **Constantes:** UPPER_SNAKE_CASE

Exemplos:

```
// Variáveis globais
float temperature = 0.0;
int soilMoistureRaw = 0;
bool ahtInitialized = false;

// Constantes
const int SOIL_DRY_VALUE = 2521;
```

```
const unsigned long sensorReadInterval = 2000;
#define SOIL_MOISTURE_PIN 34
```

11.1.2 Funções

- **Estilo:** camelCase
- **Verbos:** Usar verbos para ações (begin, read, write)
- **Descritivos:** Nomes que descrevem a ação

Exemplos:

```
void setup() { }
void loop() { }
bool begin() { }
float readLightLevel() { }
```

11.1.3 Instâncias de Objetos

- **Estilo:** camelCase ou abreviações descritivas
- **Bibliotecas:** Seguir convenção da biblioteca

Exemplos:

```
Adafruit_AHTX0 aht;          // Abreviação do sensor
BH1750 lightMeter;           // Nome descritivo
```

11.2 Estrutura de Código

11.2.1 Organização de Includes

```
// 1. Bibliotecas do sistema
#include <Arduino.h>
```

```
// 2. Headers locais
#include "credentials.h"

// 3. Bibliotecas externas (ordem alfabética)
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <BH1750.h>
```

11.2.2 Ordem de Declarações

```
// 1. Defines e constantes
#define SOIL_MOISTURE_PIN 34
const int SOIL_DRY_VALUE = 2521;

// 2. Instâncias de objetos
Adafruit_AHTX0 aht;
BH1750 lightMeter;

// 3. Variáveis globais (agrupadas por função)
// - Controle
unsigned long lastSensorRead = 0;
// - Dados dos sensores
float temperature = 0.0;
// - Status
bool ahtInitialized = false;
```

11.3 Comentários

11.3.1 Comentários de Cabeçalho

```
// Configuração do Sensor de Umidade do Solo Capacitivo
#define SOIL_MOISTURE_PIN 34  // Pino ADC do ESP32 (GPIO34)

// Calibração do sensor de umidade do solo
// Valores calibrados para este sensor específico
const int SOIL_DRY_VALUE = 2521;    // Valor ADC quando o solo está seco (no ar) -
CALIBRADO
```

11.3.2 Comentários Inline

```
// Lê sensores a cada 2 segundos
if (millis() - lastSensorRead >= sensorReadInterval) {
  // Lê sensor AHT20/AHT21 (Temperatura e Umidade)
  if (ahtInitialized) {
    // Processamento...
  }
}
```

11.3.3 Comentários de Seção

```
// ========== INICIALIZAÇÃO ==========

// ========== LEITURA DE SENSORES ==========

// ========== COMUNICAÇÃO ==========
```

11.4 Tratamento de Erros

11.4.1 Verificação de Inicialização

```
// Padrão: verificar retorno e definir flag
if (aht.begin()) {
  Serial.println("[OK] Sensor AHT20/AHT21 inicializado com sucesso.");
  ahtInitialized = true;
} else {
  Serial.println("[FALHA] Sensor AHT20/AHT21 não inicializado.");
  ahtInitialized = false;
}
```

11.4.2 Verificação de Conexão

```
// Padrão: Verificar antes de usar
if (Blynk.connected()) {
  Blynk.virtualWrite(V0, temperature);
}
```

11.4.3 Validação de Dados

```
// Padrão: Limitar faixas válidas
if (soilMoisturePercent < 0) soilMoisturePercent = 0;
if (soilMoisturePercent > 100) soilMoisturePercent = 100;
```

11.5 Timing e Delays

11.5.1 Uso de millis() (Não-Bloqueante)

```
// Recomendado: usar millis() para timing não-bloqueante
if (millis() - lastSensorRead >= sensorReadInterval) {
  lastSensorRead = millis();
```

```
  // Leitura dos sensores
}

// Evitar: delay() longo no loop principal (bloqueia o sistema)
delay(2000);
```

11.5.2 Delays Aceitáveis

```
// Delays curtos em setup ou durante conexão
delay(100);       // Setup inicial
delay(500);       // Durante tentativa de conexão
delay(2000);      // Reconexão WiFi
```

11.6 Formatação

11.6.1 Indentação

- **Estilo:** 2 espaços (não tabs)
- **Consistência:** Manter indentação consistente

```
void loop() {
  if (condition) {
    // Código indentado com 2 espaços
    if (nested) {
      // Mais 2 espaços
    }
  }
}
```

11.6.2 Espaçamento

```
// Espaços ao redor de operadores
if (millis() - lastSensorRead >= sensorReadInterval) {
  temperature = temp.temperature;
}

// Espaço após vírgula
Blynk.virtualWrite(V0, temperature);
```

11.6.3 Quebras de Linha

```
// Quebras de linha para legibilidade
if (WiFi.status() == WL_CONNECTED) {
  wifiRSSI = WiFi.RSSI();

  if (Blynk.connected()) {
    Blynk.virtualWrite(V4, wifiRSSI);
  }
}
```

11.7 Boas Práticas

11.7.1 Inicialização Segura

```
// Verificar retorno da inicialização
if (aht.begin()) {
  ahtInitialized = true;
} else {
  ahtInitialized = false;
  Serial.println("Erro na inicialização");
}
```

11.7.2 Uso de Flags de Status

```
// Usar flags para rastrear estado
bool ahtInitialized = false;

// Verificar flag antes de usar
if (ahtInitialized) {
  // Usar sensor apenas se inicializado
}
```

11.7.3 Mensagens de Log

```
Serial.println("[OK] Sensor AHT20/AHT21 inicializado.");
Serial.println("[FALHA] Sensor AHT20/AHT21 não inicializado.");
Serial.print("Temperatura: ");
```

11.7.4 Constantes para Valores Mágicos

```
// Usar constantes nomeadas
const unsigned long sensorReadInterval = 2000;
if (millis() - lastSensorRead >= sensorReadInterval) { }

// Evitar valores literais sem significado (valores mágicos)
if (millis() - lastSensorRead >= 2000) { }
```

11.8 Indicadores de Status no Serial

```
Serial.println("[OK] Sensor inicializado.");
Serial.println("[FALHA] Falha ao inicializar sensor.");
Serial.println("[AVISO] Timeout na conexão Blynk.");
```

---

12. Configuração e Instalação

12.1 Pré-requisitos

1. **Software:**
   - Visual Studio Code instalado
   - Extensão PlatformIO para VSCode
   - Python 3.x (requerido pelo PlatformIO)
   - Driver USB para ESP32 (CP2102 ou CH340)
2. **Hardware:**
   - ESP32 DevKit C (ESP32-WROOM-32)
   - Sensores: AHT20/AHT21, BH1750, Sensor de umidade do solo
   - Jumpers e protoboard (opcional)
   - Cabo USB para programação
3. **Conta Blynk:**
   - Conta criada em blynk.io
   - Template ID e Auth Token obtidos

12.2 Instalação do Ambiente

1. **Instalar PlatformIO:**
   - Abrir VSCode
   - Instalar extensão “PlatformIO IDE”
   - Aguardar instalação automática das dependências
2. **Clonar/Configurar Projeto:**

   cd "C:\Users\jorge_wilker_tech\OneDrive\Documents\PlatformIO\Projects\sensores_teste"

3. **Instalar Bibliotecas:**

- As bibliotecas serão instaladas automaticamente na primeira compilação
- Ou manualmente: pio lib install

12.3 Configuração de Credenciais

Criar arquivo include/credentials.h:

```
#ifndef CREDENTIALS_H
#define CREDENTIALS_H

// Credenciais WiFi
#define WIFI_SSID "SEU_SSID_WIFI"
#define WIFI_PASSWORD "SUA_SENHA_WIFI"

// Credenciais Blynk
#define BLYNK_TEMPLATE_ID "SEU_TEMPLATE_ID"
#define BLYNK_AUTH_TOKEN "SEU_AUTH_TOKEN"

#endif
```

Importante: Este arquivo não deve ser versionado no Git (está no .gitignore).

12.4 Compilação e Upload

Via VSCode/PlatformIO:

1. Conectar ESP32 via USB
2. Compilar: Ctrl+Alt+B ou ícone de “check”
3. Upload: Ctrl+Alt+U ou ícone de “upload”
4. Monitor Serial: Ctrl+Alt+S ou ícone de “monitor”

Via Terminal:

```
cd
"C:\Users\jorge_wilker_tech\OneDrive\Documents\PlatformIO\Projects\sensores_teste"
```

```
# Compilar
pio run

# Upload
pio run --target upload

# Monitor Serial
pio device monitor
```

---

13. Guia de Deploy e Operação

13.1 Pré-Deploy

13.1.1 Checklist de Preparação

- Hardware montado e testado
- Sensores conectados corretamente
- Credenciais WiFi configuradas em credentials.h
- Credenciais Blynk configuradas (Auth Token e Template ID)
- Valores de calibração do sensor de solo ajustados
- Código compilado sem erros
- Testes básicos realizados

13.1.2 Configuração do Blynk Dashboard

1. **Criar Projeto no Blynk:**
   - Acessar blynk.io
   - Criar novo projeto
   - Selecionar hardware: ESP32
   - Anotar Auth Token e Template ID
2. **Configurar Virtual Pins:**

  - Adicionar widgets para cada Virtual Pin:
    - **V0**: Gauge ou Value Display (Temperatura, 0-50°C)
    - **V1**: Gauge ou Value Display (Umidade Ar, 0-100%)
    - **V2**: Value Display ou Chart (Luminosidade, 0-65535 lux)
    - **V3**: Gauge ou Progress Bar (Umidade Solo, 0-100%)
    - **V4**: Value Display (RSSI WiFi, -100 a 0 dBm)
3. **Salvar Template:**
  - Salvar configuração do dashboard
  - Anotar Template ID

13.2 Processo de Deploy

13.2.1 Compilação

```
cd "C:\Users\jorge_wilker_tech\OneDrive\Documents\PlatformIO\Projects\sensores_teste"
pio run
```

Verificações: - Compilação sem erros - Sem warnings críticos - Tamanho do binário dentro dos limites do ESP32

13.2.2 Upload do Firmware

1. **Conectar ESP32:**
  - Conectar via cabo USB
  - Aguardar detecção do dispositivo
2. **Upload:**

   ```
   pio run --target upload
   ```

3. **Verificar Upload:**
  - Mensagem de sucesso no terminal

- LED do ESP32 pode piscar durante upload

13.2.3 Verificação Pós-Deploy

1. **Abrir Serial Monitor:**

   pio device monitor

2. **Verificar Saída Esperada:**

   - Inicialização dos sensores ([OK] ou [FALHA])
   - Conexão WiFi ([OK] ou [FALHA])
   - Conexão Blynk ([OK] ou [FALHA])
   - Leituras dos sensores a cada 2 segundos

3. **Verificar Blynk Dashboard:**

   - Dados aparecendo nos widgets
   - Valores atualizando a cada 2 segundos
   - RSSI mostrando qualidade do sinal

13.3 Operação Normal

13.3.1 Monitoramento

Serial Monitor: - Verificar logs a cada 2 segundos - Confirmar que todos os sensores estão funcionando - Monitorar qualidade do sinal WiFi (RSSI)

Blynk Dashboard: - Visualizar dados em tempo real - Verificar atualização contínua dos valores - Monitorar tendências (se usar gráficos)

13.3.2 Manutenção Preventiva

Diária: - Verificar se sistema está operando (via Blynk ou Serial) - Confirmar que dados estão sendo atualizados

Semanal: - Verificar qualidade do sinal WiFi (RSSI) - Limpar sensores se necessário (poeira, umidade) - Verificar conexões físicas

Mensal: - Recalibrar sensor de solo se necessário - Verificar precisão dos sensores (comparar com instrumentos de referência) - Atualizar firmware se houver novas versões

---

13.4 Operação em Caso de Problemas

13.4.1 Sistema Não Inicializa

Sintomas: - Nenhuma saída no Serial Monitor - LED do ESP32 não pisca

Ações: 1. Verificar conexão USB 2. Verificar driver USB instalado 3. Tentar outro cabo USB 4. Verificar alimentação do ESP32 5. Resetar ESP32 (botão RESET)

13.4.2 Sensores Não Funcionam

Sintomas: - Mensagens “[FALHA] Falha ao inicializar sensor” no Serial

Ações: 1. Verificar conexões I2C (SDA=GPIO21, SCL=GPIO22) 2. Verificar alimentação dos sensores (3.3V, GND) 3. Verificar se sensores não estão danificados 4. Testar sensores isoladamente 5. Verificar endereços I2C (usar scanner I2C)

13.4.3 WiFi Não Conecta

Sintomas: - Mensagem “[FALHA] Falha na conexão WiFi!”

Ações: 1. Verificar credenciais em credentials.h 2. Verificar se roteador está funcionando 3. Verificar se rede WiFi está no alcance 4. Verificar se roteador suporta 2.4 GHz (802.11 b/g/n) 5. Verificar firewall do roteador

---

13.4.4 Blynk Não Conecta

Sintomas: - Mensagem “[FALHA] Falha na conexão Blynk!”

Ações: 1. Verificar Auth Token em credentials.h 2. Verificar Template ID em credentials.h 3. Verificar conexão com internet (WiFi conectado) 4. Verificar status do servidor Blynk 5. Aguardar até 30 segundos (timeout configurado) 6. Sistema continuará funcionando localmente (Serial Monitor)

13.5 Atualização de Firmware

13.5.1 Processo de Atualização

1. **Fazer Backup:**
   - Anotar valores de calibração atuais
   - Anotar configurações do Blynk
2. **Atualizar Código:**
   - Fazer pull das atualizações (se usar Git)
   - Ou editar código conforme necessário
3. **Recompilar e Fazer Upload:**

   pio run --target upload

4. **Verificar Funcionamento:**
   - Monitorar Serial Monitor
   - Verificar Blynk Dashboard

13.5.2 Rollback (Reverter)

Se a atualização causar problemas:

1. **Reverter para Versão Anterior:**

- Usar Git: git checkout <commit-anterior>
- Ou restaurar código manualmente

2. **Recompilar e Fazer Upload:**

   pio run --target upload

13.6 Descomissionamento

13.6.1 Remoção Segura

1. **Desconectar Alimentação:**
   - Desconectar USB
   - Remover alimentação externa (se houver)
2. **Remover Sensores:**
   - Desconectar sensores I2C
   - Desconectar sensor de solo
3. **Armazenamento:**
   - Guardar em local seco
   - Proteger contra umidade e poeira

13.7 Logs e Diagnóstico

13.7.1 Logs do Serial Monitor

Localização: Saída do Serial Monitor (115200 baud)

Conteúdo: - Status de inicialização - Leituras dos sensores (a cada 2 segundos) - Status de conexão WiFi e Blynk - Mensagens de erro

Captura de Logs:

```
# Salvar logs em arquivo
pio device monitor > logs.txt
```

13.7.2 Logs do Blynk

Localização: Dashboard Blynk (gráficos e histórico)

Conteúdo: - Histórico de valores dos sensores - Gráficos de tendências - Eventos de conexão/desconexão

Acesso: - Via app Blynk (mobile) - Via web dashboard (blynk.io)

---

14. Protocolos e Comunicação

14.1 Protocolo I2C

Configuração: - **Velocidade:** Padrão (100 kHz) - **Pinos:** SDA=GPIO21, SCL=GPIO22 - **Endereços:** - AHT20/AHT21: 0x38 - BH1750: 0x23

Inicialização:

```
Wire.begin();  // Inicializa I2C com pinos padrão
```

14.2 Protocolo WiFi

Especificações: - **Padrão:** IEEE 802.11 b/g/n - **Frequência:** 2.4 GHz - **Modo:** Station (STA) - cliente WiFi - **Reconexão:** Automática em caso de perda de conexão

Configuração:

```
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

14.3 Protocolo Blynk

Configuração: - **Servidor:** Blynk Cloud (padrão) - **Protocolo:** TCP/IP sobre WiFi - **Autenticação:** Auth Token + Template ID - **Timeout de Conexão:** 30 segundos

Virtual Pins Configurados:

- **V0 (Temperatura):** Float - Unidade: °C
- **V1 (Umidade do Ar):** Float - Unidade: % RH
- **V2 (Luminosidade):** Float - Unidade: lux
- **V3 (Umidade do Solo):** Float - Unidade: %
- **V4 (Sinal WiFi - RSSI):** Integer - Unidade: dBm

Envio de Dados:

```
Blynk.virtualWrite(V0, temperature);      // Temperatura
Blynk.virtualWrite(V1, humidity);         // Umidade do ar
Blynk.virtualWrite(V2, lightLevel);       // Luminosidade
Blynk.virtualWrite(V3, soilMoisturePercent); // Umidade do solo
Blynk.virtualWrite(V4, wifiRSSI);         // RSSI WiFi
```

---

15. Calibração e Configuração

15.1 Calibração do Sensor de Umidade do Solo

O sensor de umidade do solo capacitivo requer calibração específica para cada unidade. Os valores de calibração estão definidos no código:

```
const int SOIL_DRY_VALUE = 2521;    // Valor ADC quando seco (no ar)
const int SOIL_WET_VALUE = 1200;    // Valor ADC quando molhado (na água)
```

Processo de Calibração:

1. **Medição do Valor Seco:**
   - Colocar sensor no ar (sem contato com água)
   - Ler valor ADC via Serial Monitor
   - Atualizar SOIL_DRY_VALUE
2. **Medição do Valor Molhado:**

  - Submergir sensor em água
  - Ler valor ADC via Serial Monitor
  - Atualizar SOIL_WET_VALUE
3. **Conversão para Porcentagem:**

```
soilMoisturePercent = map(soilMoistureRaw, SOIL_DRY_VALUE, SOIL_WET_VALUE, 0, 100);
// Limita entre 0 e 100%
if (soilMoisturePercent < 0) soilMoisturePercent = 0;
if (soilMoisturePercent > 100) soilMoisturePercent = 100;
```

15.2 Configuração de Intervalos

Intervalos Configurados:

- **sensorReadInterval:** 2000 ms - Intervalo entre leituras dos sensores
- **connectionCheckInterval:** 2000 ms - Intervalo de verificação de conexão
- **blynkTimeout:** 30000 ms - Timeout para conexão Blynk

15.3 Configuração do Blynk Dashboard

Widgets Recomendados:

1. **V0 - Temperatura:**
  - Widget: Gauge ou Value Display
  - Range: 0-50°C (ajustável conforme aplicação)
2. **V1 - Umidade do Ar:**
  - Widget: Gauge ou Value Display
  - Range: 0-100%
3. **V2 - Luminosidade:**
  - Widget: Value Display ou Chart
  - Range: 0-65535 lux
4. **V3 - Umidade do Solo:**
  - Widget: Gauge ou Progress Bar

  - Range: 0-100%
5. **V4 - RSSI WiFi:**
  - Widget: Value Display
  - Range: -100 a 0 dBm

---

16. Especificações Técnicas Detalhadas

16.1 Performance do Sistema

Taxa de Amostragem: 0.5 Hz (1 leitura a cada 2 segundos)
Latência de Transmissão: < 1 segundo (depende da rede)
Tempo de Inicialização: ~5-10 segundos
Tempo de Conexão WiFi: < 10 segundos (típico)
Tempo de Conexão Blynk: < 30 segundos (máximo)

16.2 Consumo de Energia

Modo Ativo (leituras): ~80-120 mA @ 3.3V
Modo WiFi Conectado: ~80-100 mA @ 3.3V
Modo Deep Sleep: ~10 µA (não implementado)

Nota: Valores aproximados, variam conforme condições de rede e carga de processamento.

16.3 Precisão e Resolução

AHT20/AHT21 - Temperatura: - Precisão: ±0.3°C - Resolução: 0.01°C

AHT20/AHT21 - Umidade: - Precisão: ±2% RH - Resolução: 0.01% RH

BH1750 - Luminosidade: - Precisão: ±20% (típico) - Resolução: 1 lux

Sensor de Solo - Umidade: - Precisão: Depende da calibração - Resolução: 0.1% (após calibração)

16.4 Faixas de Operação

Temperatura Ambiente: 0°C a 70°C (recomendado)

Umidade Relativa: 20% RH a 90% RH

Tensão de Alimentação: 3.0V a 3.6V

Distância WiFi: Depende do roteador (típico: < 50m)

---

17. Diagramas de Conexão

17.1 Diagrama Completo de Conexões

```
                    ESP32 DevKit C
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  [3.3V] ──┬──────────────────────────────────────────┐   │
│           │                                          │   │
│  [GND]  ──┼──────────────────────────────────────────┤   │
│           │                                          │   │
│  [GPIO21] ──┼──── SDA (I2C)                          │   │
│           │                                          │   │
│  [GPIO22] ──┼──── SCL (I2C)                          │   │
│           │                                          │   │
│  [GPIO34] ──┼──── ADC (Sensor Solo)                  │   │
│                                                      │   │
└──────────────────────────────────────────────────────┘
              │         │         │
    ┌─────────┘         │         └─────────┐
    │                   │                   │
    ▼                   ▼                   ▼
┌─────────┐         ┌─────────┐         ┌─────────┐
│ AHT20/  │         │ BH1750  │         │ Sensor  │
│ AHT21   │         │         │         │ Solo    │
│         │         │         │         │         │
│ SDA ────┼─────────┼──── SDA │         │ AOUT ───┼──┐
│ SCL ────┼─────────┼──── SCL │         │         │  │
│ VCC ────┼─────────┼──── VCC │         │ VCC ────┼──┤
│ GND ────┼─────────┼──── GND │         │ GND ────┼──┤
└─────────┘         └─────────┘         └─────────┘
```

17.2 Pinout do ESP32 DevKit C

```
                    (Vista Superior / Top View)
                    ┌──────────────────────────┐
               5V   │ o                    o   │   23
              VIN   │ o                    o   │   22  (SCL) <--- [I2C]
               13   │ o                    o   │   TX
               12   │ o                    o   │   RX
               14   │ o                    o   │   21  (SDA) <--- [I2C]
               27   │ o                    o   │   GND <--- [GND]
               26   │ o                    o   │   19
               25   │ o                    o   │   18
               33   │ o                    o   │   5
               32   │ o                    o   │   17
               35   │ o                    o   │   16
(Sensor Solo)  34   │ o <---               o   │   4
              VN    │ o                    o   │   0
              VP    │ o                    o   │   2
              EN    │ o                    o   │   15
              GND   │ o                    o   │   CLK
                    └──────────[USB]────────────┘
```

Nota: O layout acima refere-se ao ESP32 DevKit C (WROOM-32). Os pinos GPIO 21 (SDA), GPIO 22 (SCL) e GPIO 34 (Sensor Solo) estão marcados. Verifique as etiquetas impressas na sua placa para confirmar a localização exata dos pinos.

---

18. Troubleshooting

18.1 Problemas Comuns e Soluções

Sensor AHT20/AHT21 não inicializa

Sintomas: - Mensagem “[FALHA] Falha ao inicializar sensor AHT20/AHT21” no Serial Monitor

Possíveis Causas: 1. Conexões I2C incorretas 2. Alimentação insuficiente 3. Sensor danificado 4. Endereço I2C incorreto

Soluções: 1. Verificar conexões: SDA=GPIO21, SCL=GPIO22, VCC=3.3V, GND=GND 2. Verificar alimentação (3.3V estável) 3. Testar sensor em outro dispositivo 4. Verificar endereço I2C com scanner I2C

Sensor BH1750 não inicializa

Sintomas: - Mensagem “[FALHA] Falha ao inicializar sensor BH1750” no Serial Monitor

Possíveis Causas: 1. Conexões I2C incorretas 2. Conflito de endereço I2C 3. Sensor danificado

Soluções: 1. Verificar conexões I2C compartilhadas 2. Verificar se endereços I2C são diferentes (AHT20: 0x38, BH1750: 0x23) 3. Testar sensor isoladamente

WiFi não conecta

Sintomas: - Mensagem “[FALHA] Falha na conexão WiFi!” após 10 segundos

Possíveis Causas: 1. SSID ou senha incorretos 2. Sinal WiFi fraco 3. Roteador não suporta 2.4 GHz 4. Firewall bloqueando conexão

Soluções: 1. Verificar credenciais em credentials.h 2. Verificar força do sinal (RSSI) 3. Confirmar que roteador suporta 802.11 b/g/n (2.4 GHz) 4. Verificar configurações de firewall

Blynk não conecta

Sintomas: - Mensagem “[FALHA] Falha na conexão Blynk!” após 30 segundos

Possíveis Causas: 1. Auth Token incorreto 2. Template ID incorreto 3. Sem conexão com internet 4. Servidor Blynk inacessível

Soluções: 1. Verificar Auth Token em credentials.h 2. Verificar Template ID em credentials.h 3. Testar conexão com internet (ping) 4. Verificar status do servidor Blynk

Valores incorretos do sensor de solo

Sintomas: - Valores de umidade do solo fora da faixa esperada (0-100%)

Possíveis Causas: 1. Calibração incorreta 2. Sensor danificado 3. Conexão analógica instável

Soluções: 1. Recalibrar sensor (ver seção 7.1) 2. Verificar conexão no GPIO 34 3. Testar sensor com multímetro

18.2 Diagnóstico via Serial Monitor

Saída esperada (normal):

```
=== Sistema de Monitoramento ESP32 ===

Sensor de Umidade do Solo Capacitivo:
  Configurado no pino GPIO34
  [OK] Pronto para leitura analógica

Inicializando sensor AHT20/AHT21...
[OK] Sensor AHT20/AHT21 inicializado com sucesso.

Inicializando sensor BH1750...
[OK] Sensor BH1750 inicializado com sucesso.

Conectando ao WiFi...
```

SSID: SEU_SSID
[OK] WiFi conectado com sucesso.
IP Address: 192.168.1.100
RSSI: -45 dBm

Conectando ao Blynk...
[OK] Blynk conectado com sucesso.

=== Sistema Pronto ===
Virtual Pins configurados:
  V0 - Temperatura (°C)
  V1 - Umidade do Ar (%)
  V2 - Luminosidade (lux)
  V3 - Umidade do Solo (%)
  V4 - Sinal WiFi (dBm)

--- Leituras dos Sensores ---
Temperatura: 25.3 °C
Umidade Ar: 60.5 %
Luminosidade: 350 lux
Umidade Solo: 45 % (ADC: 1860)
Sinal WiFi: -45 dBm (Excelente)

18.3 Códigos de Erro

- **"[FALHA] Falha ao inicializar sensor AHT20/AHT21":** Sensor I2C não detectado -> Verificar conexões I2C
- **"[FALHA] Falha ao inicializar sensor BH1750":** Sensor I2C não detectado -> Verificar conexões I2C
- **"[FALHA] Falha na conexão WiFi":** WiFi não conectou -> Verificar credenciais e sinal

- **“[FALHA] Falha na conexão Blynk”:** Blynk não conectou -> Verificar token e internet
- **“WiFi desconectado! Tentando reconectar…”:** Perda de conexão WiFi -> Sistema tenta reconectar automaticamente

---

19. Referências e Documentação Adicional

19.1 Documentação Oficial

- **ESP32:** Espressif ESP32 Documentation
- **PlatformIO:** PlatformIO Documentation
- **Blynk:** Blynk Documentation
- **AHT20/AHT21:** Adafruit AHTX0 Library
- **BH1750:** BH1750 Library

19.2 Documentação do Projeto

- README.md - Documentação geral do projeto

---

20. Histórico de Versões

- **1.0 (Janeiro 2026):** Documentação técnica inicial baseada no estado final do projeto

---

21. Licença

Este projeto é licenciado sob GNU General Public License v3.0 (GPL-3.0).

Para mais detalhes, consulte o arquivo LICENSE ou visite https://www.gnu.org/licenses/gpl-3.0.html.

Documentação Técnica - Sistema de Monitoramento IoT com ESP32

Última atualização: Janeiro 2026